

Лабораторна робота №3

Згорткова нейронна мережа (CNN)

Мета роботи: ознайомлення зі створенням та навчанням згорткової нейронної мережі (CNN) на основі використання бібліотек **TensorFlow** та **Keras**, а також візуалізації роботи CNN на основі застосування функцій бібліотеки **Matplotlib**.

Методичні рекомендації

Для створення штучної згорткової нейронної мережі будемо використовувати відкриту нейромережеву бібліотеку **Keras** на мові Python. Вона є надбудовою над фреймворком TensorFlow та спрощує з ним роботу.

TensorFlow — відкрита програмна бібліотека для машинного навчання, розроблена компанією Google для вирішення задач побудови та навчання нейронних мереж.

1. Початкові налаштування.

Спочатку необхідно завантажити бібліотеку *TensorFlow 2.0.0* (або вище), а потім - *Keras*.

Далі можливо буде необхідно перезавантажити стандартну бібліотеку *numpy*. Наприклад, версія *numpy 1.16.4* працює з *Python 3.6.8* (якщо ви працюєте саме з цією версією).

```
pip install tensorflow==2.0.0
pip install keras
pip uninstall numpy
pip install numpy==1.16.4
```

2. Завантаження бази даних MNIST, яка є вбудованою в Keras:

```
# Підключаємо датасет.
from keras.datasets import mnist
#Завантажуємо дані x_train и x_test які містять двовимірний масив із зображенням
цифр
# x_test, y_test масив з даними для перевірки.
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# Трансформуємо з двовимірного масиву у тривимірний масив (28x28x1 канал)
x_train = x_train.reshape(60000,28,28,1)
x_test = x_test.reshape(10000,28,28,1)
```

У цьому коді функція Numpy **reshape()** змінює форму масиву без зміни даних.

3. Перетворення тестових даних.

```
from keras.utils import to_categorical
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

Функція `keras.utils.to_categorical(y, num_classes=None, dtype='float32')` перетворює вектор класу (цілі числа) у бінарну матрицю класів.

4. Побудова моделі мережі.

```
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten
# створення моделі
model = Sequential()
# Додавання шару
model.add(Conv2D(64, kernel_size=3, activation='relu', input_shape=(28,28,1)))
```

Перший шар буде згортковий двовимірний (*Conv2D*). Ці згорткові шари будуть працювати з вхідними зображеннями, які розглядаються як двовимірні матриці. `kernel_size=3` — розмір ядра 3x3. Функція активації '*relu*' (Rectified Linear Activation), `64` — це кількість ядер згортки (кількість фільтрів).

```
# Другий згортковий шар
model.add(Conv2D(32, kernel_size=3, activation='relu'))
# Створюємо вектор для повнозв'язної мережі
model.add(Flatten())
```

Функція Keras **Flatten()** — створює шар, який перетворює 2D-дані у вектор.

```
# Створюємо одношаровий перцептрон
model.add(Dense(10, activation='softmax'))
```

Де параметр **Dense()** — повнозв'язний шар, кількість виходів — `10`, функція активації '*softmax*'.

Далі треба зробити компіляцію моделі. Компіляція моделі використовує три параметри: **оптимізатор, функція втрат та метрика**.

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

Оптимізатор вагових коефіцієнтів `optimizer='adam'` має параметр `'adam'`. Оптимізатор — це метод досягнення найкращих результатів, допомога у прискоренні навчання. Adam - це метод стохастичної оптимізації. Функція втрат: `loss='categorical_crossentropy'` категоріальна перехрестна ентропія (categorical crossentropy CCE). Функція втрат — це міра того, наскільки добре ваша модель прогнозування передбачає результат, що очікується (або значення). Останній параметр — метрика, яка визначає метод визначення точності моделі (у данному випадку - ймовірність правильної класифікації). У загальному вигляді: `metrics={'output_a': 'accuracy'}`.

5. Навчання мережі:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=1)
print(hist.history)
```

Де `epochs=1` кількість епох навчання, `validation_data=(x_test, y_test)` — перевірочні дані. Результат тренування наведено на скріншоті (рис.1).

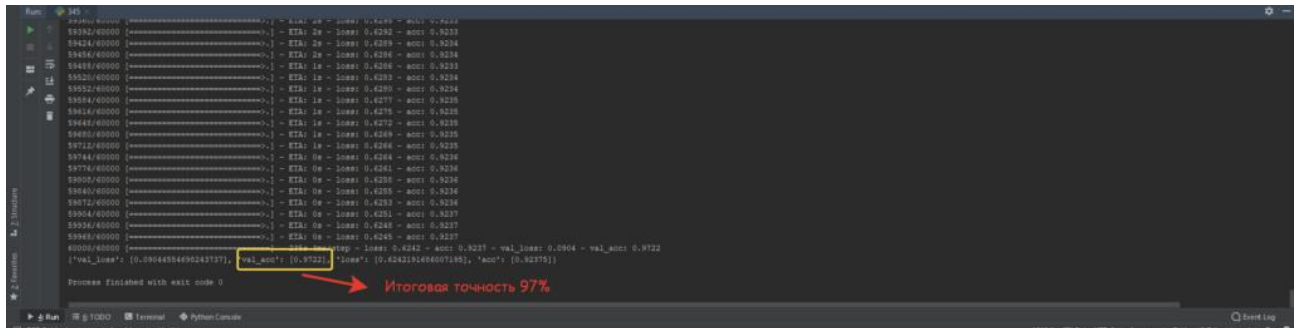


Рис.1. Результат роботи мережі

6. Візуалізація, зберігання моделі.

Для візуалізації результатів навчання мережі побудуємо графіки. Для цього використовуємо метод `fit()`, який повертає історію навчання (кількість епох = 5). Результат візуалізації наведено на рис.2 та рис.3.

```
import matplotlib.pyplot as plt
history = model.fit(x_train, y_train, validation_data=(x_test, y_test),
epochs=5)
```

```
# Plot training & validation accuracy values
plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()

#
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

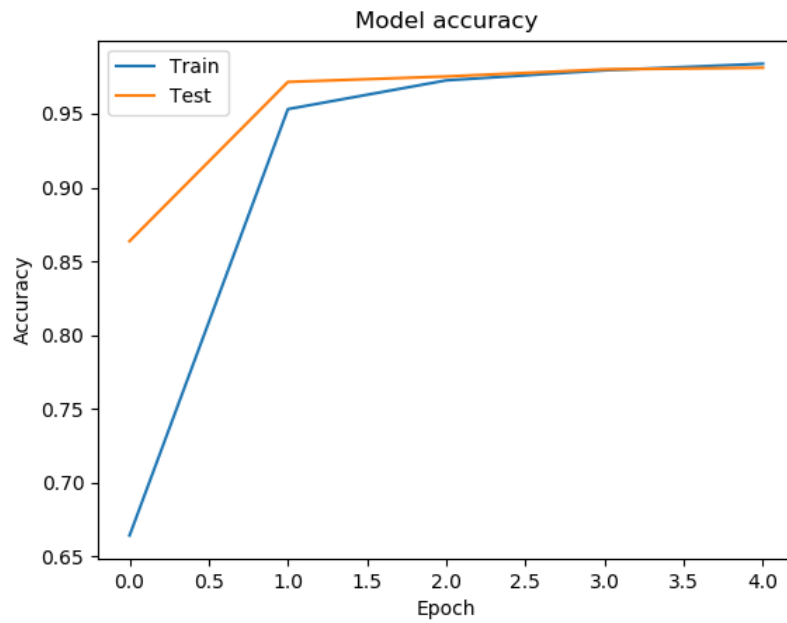


Рис.2. Графік точності навчання

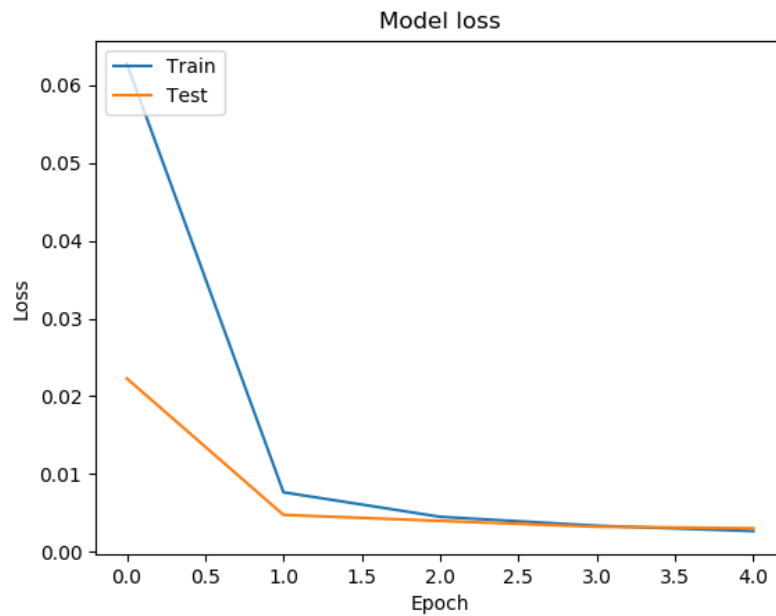


Рис. 3. Графік кількості втрат (помилки)

Для зберігання моделі Keras в одному файлі HDF5, який буде містити:

- архітектуру моделі, яка дозволяє відтворити модель;
- вагові коефіцієнти моделі;
- конфігурацію навчання (втрати, оптимізатор);
- стан оптимізатору, який дозволяє відновити тренування саме там, де ви зупинилися,

можна використовувати функцію `model.save(filepath:`

```
from keras.models import load_model

model.save('my_model.h5') # Створення HDF5 файлу 'my_model.h5'
del model # Видалення моделі.

# Завантаження моделі знову
model = load_model('my_model.h5')
```

7. Використання GPU.

Якщо ви працюєте з **TensorFlow** або **CNTK** backends (The Microsoft Cognitive Toolkit), ваш код автоматично запускається на GPU, якщо знайдено якийсь доступний GPU.

- `tensorflow` — Latest stable release for CPU-only
- `tensorflow-gpu` — Latest stable release with GPU support (*Ubuntu and Windows*)

Індивідуальні завдання.

1. Додати шар Макспулінгу `MaxPooling2D` (але його спочатку треба імпортувати з Keras) та запустити 10, 50, 100 епох навчання.
2. Змінити оптимізатор на `'sgd'` – стохастичний градієнтний спуск.
3. Змінити функцію помилок (втрат) на `'mean_squared_error'` – середньоквадратичну помилку.
4. Провести тренування на датасеті **MNIST** та візуалізувати результати навчання.
5. Зробити оцінку отриманих результатів та висновки щодо ефективності застосованих гіперпараметрів: архітектури CNN (додаткового шару), кількості епох навчання, обраного оптимізатору та функції помилок.

Зміст звіту.

1. Код програми з коментарями.
2. Графіки результатів тренування мережі.
3. Оцінка результатів та висновки.